

AT32 BLDC Hall Quick Start Guide

Introduction

This application note describes the standard operating procedure (SOP) for quickly debugging the operation of a BLDC motor with Hall sensors using a motor development board. Artery offers many motor development boards, and this document uses the AT-MOTOR-EVB as an example for illustration..

AT-MOTOR-EVB Applicable products:

Part number	AT32F402/405 series
	AT32F413 series
	AT32F415 series
	AT32F421 series
	AT32F422/426 series
	AT32F425 series
	AT32L021 series
	AT32F45x series
	AT32M412/M416 series

Contents

1	Motor control library algorithm	6
2	Environment requirements	7
	2.1 Hardware requirements	7
	2.2 Software requirements	9
3	Quick start guide	13
	3.1 Modify motor control mode and parameters	14
	3.2 Connection settings	17
	3.3 Six-step square wave open loop control	17
	3.4 Hall phase sequence self-learning.....	19
	3.5 Current sampling point	20
	3.6 Auto identification and tuning of parameters.....	22
	3.7 IQ tune	25
	3.8 Speed loop control	27
	3.9 Speed voltage control.....	29
	3.10 External voltage control/software command control	32
4	Revision history.....	35

List of tables

Table 1. Flash memory size and ROM configuration	9
Table 2. Macro definition setting (internal crystal)	14
Table 3. Macro definition setting (EVB version).....	14
Table 4. Macro definition setting (upper-side PWM complementarity).....	14
Table 5. Macro definition setting (low-side MOS inverted output).....	14
Table 6. Macro definition setting (EMF voltage compensate)	14
Table 7. Macro definition setting (sensored/sensorless method).....	15
Table 8. Macro definition setting (without current loop).....	15
Table 9. Macro definition setting (low-speed voltage control)	15
Table 10. Macro definition setting (auto identification of motor winding parameters)	15
Table 11. Macro definition setting (Artery motor control UI tool)	15
Table 11. Macro definition setting (Artery motor control UI tool)	15
Table 12. Motor parameter definition	16
Table 13. Document revision history.....	35

List of figures

Figure 1. AT-MOTOR-EVB evaluation board.....	7
Figure 2. AT32F413RCT7 ROM settings (Keil)	10
Figure 3. AT32F413RCT7 ROM settings (AT32IDE).....	11
Figure 4. AT32F413RCT7 ROM settings (IAR)	12
Figure 5. Connection setting.....	17
Figure 6. Select open loop control.....	17
Figure 7. Open loop control parameters (BLDC).....	18
Figure 8. Open loop control	18
Figure 9. Macro definition of open loop control parameters.....	18
Figure 10. Hall phase sequence self-learning	19
Figure 11. Macro definition of Hall self-learning	20
Figure 12. Current sampling point signal measurement position (AT32-Motor-EVB2.0).....	21
Figure 13. Current sampling delay measurement waveform	22
Figure 14. Macro definition of current sampling point	22
Figure 15. Macro definition of motor nominal current.....	23
Figure 16. Auto identification of winding parameters	23
Figure 17. Macro definitions of winding parameters	23
Figure 18. Motor rated voltage.....	24
Figure 19. Current PI controller parameter auto tuning.....	24
Figure 20. Macro definition of current PI controller parameters.....	24
Figure 21. Schematic diagram of step current	25
Figure 22. Select IQ tune.....	25
Figure 23. PID parameters and step current parameters.....	25
Figure 24. Set channel monitoring parameters (IQ tune).....	26
Figure 25. IQ tune waveform	26
Figure 26. Macro definition of current PI controller parameters.....	27
Figure 27. PID parameters, acceleration and deceleration.....	27
Figure 28. Set target speed	27
Figure 29. Set channel monitoring parameters (speed control).....	28
Figure 30. Waveform in speed control mode.....	28
Figure 31. Definition of low-speed voltage control	29
Figure 32. Low-speed voltage control parameter definition	29
Figure 33. Speed voltage controller PID parameter setting	30

Figure 34. Set target speed	30
Figure 35. Set channel monitoring parameters (speed control).....	30
Figure 36. Waveform in speed voltage control mode.....	31
Figure 37. External voltage control – potentiometer	32
Figure 38. Control source definition (software control (upper)/external voltage control (lower)).....	32
Figure 39. Speed control mode (external voltage control)	33
Figure 40. Torque control mode (external voltage control).....	33
Figure 41. Set target speed in software control mode	34

1 Motor control library algorithm

This application note mainly covers the following content:

Target motor type: Three-phase permanent magnet synchronous motor (brushless DC motor, BLDC)

Control methods:

- 120° square wave commutation

Three-phase PWM method:

- 120° conduction PWM control

Phase current sensing method:

- 1-shunt current sensing and reconstruction method

Rotor position detection mode:

- Hall-effect position sensor

Sensor-based 120° square wave commutation mode:

- 120° square wave voltage control
- Torque control (120° square wave current control)
- Speed control
- Forward reverse control

Auto tuning:

- Hall state self-learning (for hall sensor only)
- Automatic identification of motor winding parameters
- Current PI controller parameter auto tuning

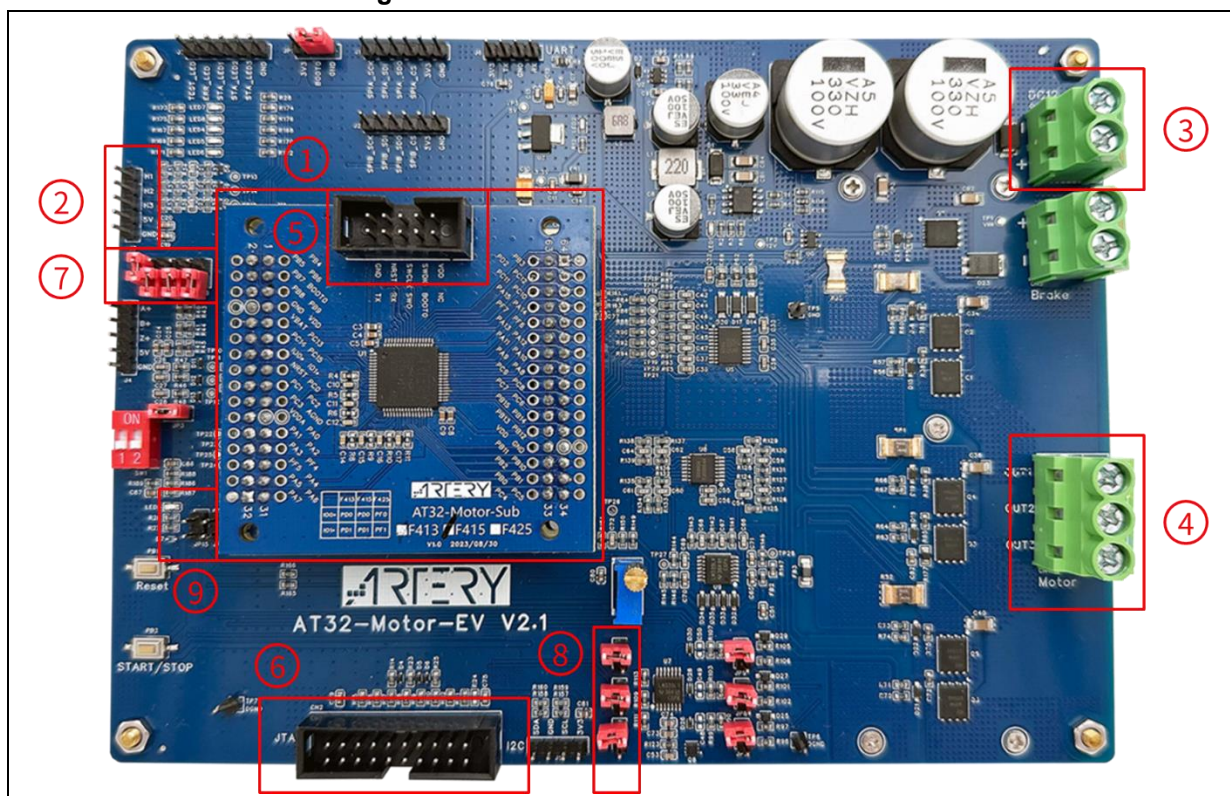
2 Environment requirements

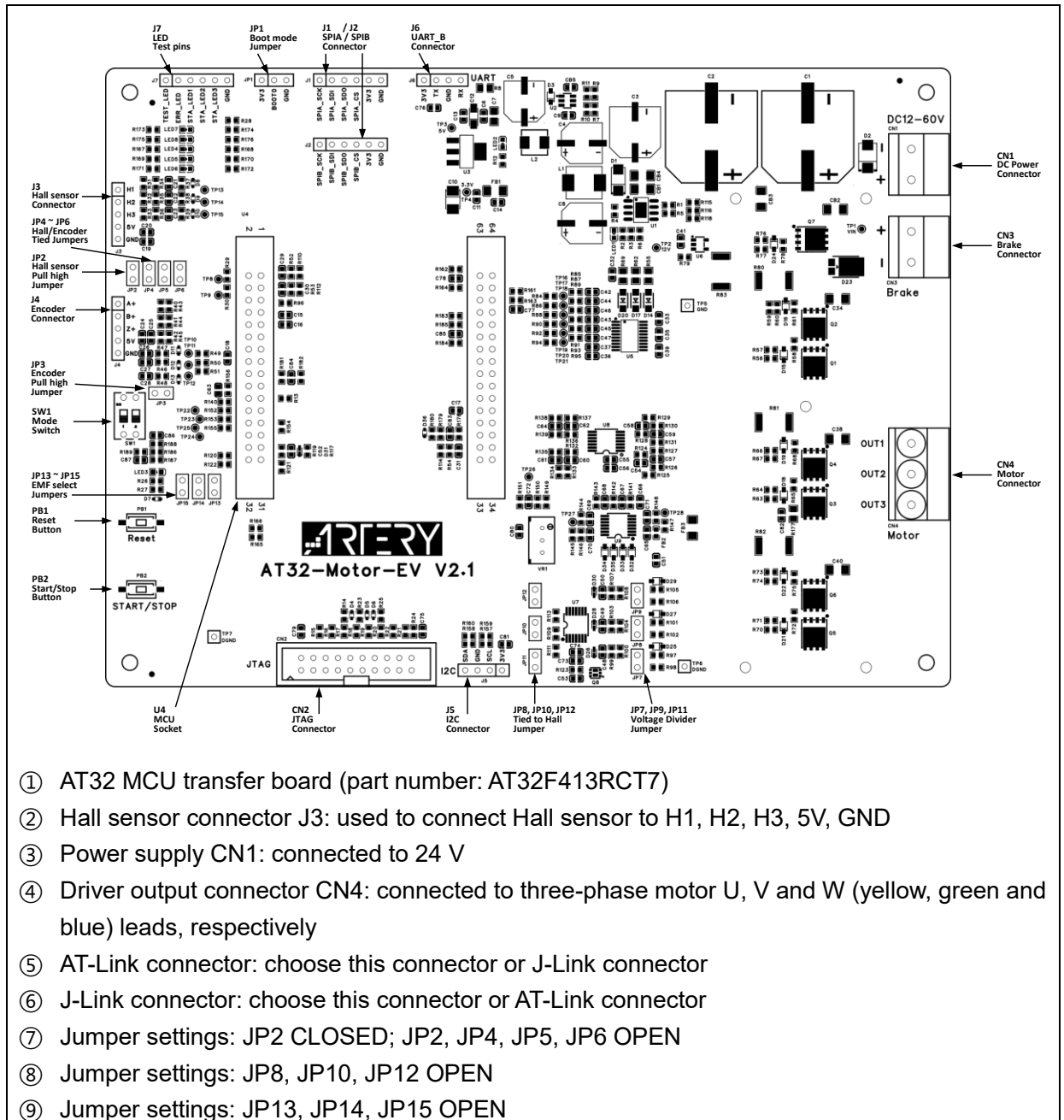
2.1 Hardware requirements

A PMSM (BLDC) motor, AT-Link or third-party debugger and a motor control board (AT-MOTOR-EVB) are required. Refer to *UM0011 AT32_LV_Motor_Control_EVB_User_Manual* for relevant hardware configurations.

- PMSM (BLDC) motor
- Debugger
- Motor control board: AT-MOTOR-EVB (including AT32F413RCT7 transfer board)

Figure 1. AT-MOTOR-EVB evaluation board





2.2 Software requirements

- 1) Open the “bldc_1shunt_hall_sensor” demo project;
- 2) Run ArteryMotorMonitor.exe directly (installation not required);
- 3) In Keil, go to **Options - Read/Only Memory Areas** to set ROM according to the Flash memory size of each AT32 MCU (see Table 1). For example, for AT32F413RCT7 with a 256KB Flash memory, its IROM1 start address is 0x8000000 with a size of 0x3F800, whereas its IROM2 start address is 0x803F800 with a size of 0x800 (see Figure 2). In AT32IDE, modify *ld file according to the Flash memory size of each AT32 MCU (see Figure 3). In IAR Systems, modify *icf file according to the Flash memory size of each AT32 MCU (see Figure 4).

Table 1. Flash memory size and ROM configuration

Flash size	4032K	1024K	512K	448K	256K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x3EF000	0xFF800	0x7F800	0x6F000	0x3F800
IROM2(address)	0x83EF000	0x80FF800	0x807F800	0x0806F000	0x803F800
IROM2(size)	0x1000	0x800	0x800	0x1000	0x800

Flash size	128K	64K	32K	16K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x1FC00	0x0FC00	0x07C00	0x03C00
IROM2(address)	0x801FC00	0x800FC00	0x8007C00	0x8003C00
IROM2(size)	0x400	0x400	0x400	0x400

Note [1]: Keil compiler version 5 is recommended since AT32 BSP source code does not support V6.15 compiler.

Figure 2. AT32F413RCT7 ROM settings (Keil)

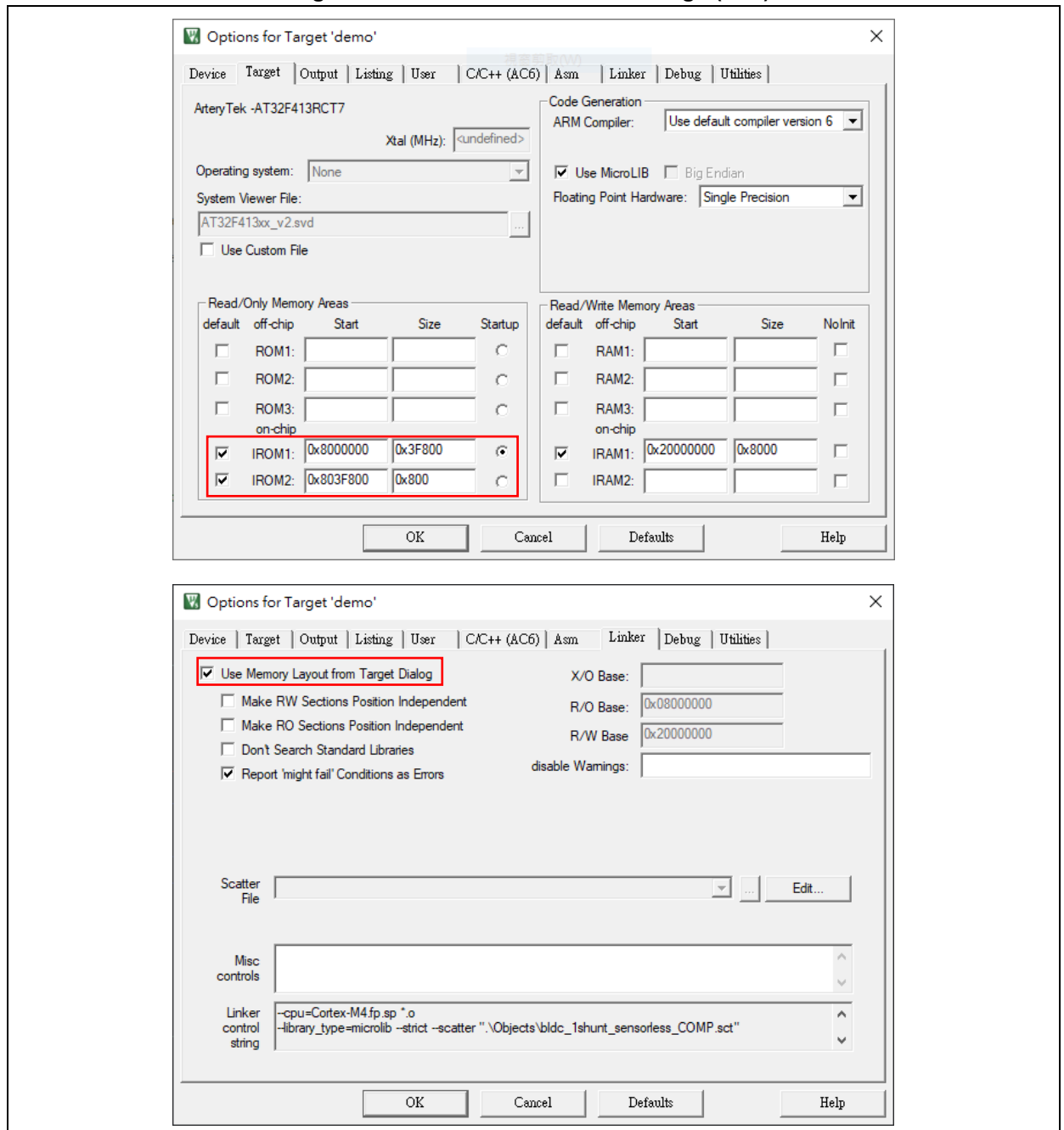


Figure 3. AT32F413RCT7 ROM settings (AT32IDE)

```

AT32F413xC_FLASH.ld X
25 _estack = 0x20008000; /* end of RAM */
26
27 /* Generate a link error if heap and stack don't fit into RAM */
28 _Min_Heap_Size = 0x200; /* required amount of heap */
29 _Min_Stack_Size = 0x400; /* required amount of stack */
30
31 /* Specify the memory areas */
32 MEMORY
33 {
34     FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 0x3F800
35     MC_DATA (r)      : ORIGIN = 0x0803F800, LENGTH = 0x800
36     RAM (xrw)        : ORIGIN = 0x20000000, LENGTH = 32K
37 }
38
39 /* Define output sections */
40 SECTIONS
41 {
42     /* The startup code goes first into FLASH */
43     .isr_vector :
44     {
45         . = ALIGN(4);
46         KEEP(*(.isr_vector)) /* Startup code */
47         . = ALIGN(4);
48     } >FLASH
49
50     .mc_data :
51     {
52         . = ALIGN(4);
53         . = ORIGIN(MC_DATA);
54         _MC_VectStoreAddr1 = .;
55         *(.MC_VectStoreAddr1);
56         . = ALIGN(4);
57         . = ORIGIN(MC_DATA) + 800;
58         _MC_VectStoreAddr2 = .;
59         *(.MC_VectStoreAddr2);
60         . = ALIGN(4);
61     } >MC_DATA
62
63     /* The program code and other data goes into FLASH */
64     .text :
65     {
66         . = ALIGN(4);
67         *(.text)           /* .text sections (code) */
68         *(.text*)          /* .text* sections (code) */
69         *(.glue_7)         /* glue arm to thumb code */
70         *(.glue_7t)        /* glue thumb to arm code */
71         *(.eh_frame)
72

```

Figure 4. AT32F413RCT7 ROM settings (IAR)

```

1  /*###ICF### Section handled by ICF editor, don't touch! ****/
2  /*-Editor annotation file-*/
3  /* IcfEditorFile="$TOOLKIT_DIR$\config\ide\IcfEditor\cortex_v1_0.xml" */
4  /*-Specials-*/
5  define symbol __ICFEDIT_intvec_start__ = 0x08000000;
6  /*-Memory Regions-*/
7  define symbol __ICFEDIT_region_ROM_start__ = 0x08000000;
8  define symbol __ICFEDIT_region_ROM_end__ = 0x0803F800 - 1;
9  define symbol __ICFEDIT_region_ROM1_start__ = 0x0803F800;
10 define symbol __ICFEDIT_region_ROM1_end__ = (__ICFEDIT_region_ROM1_start__ + 800 - 1);
11 define symbol __ICFEDIT_region_ROM2_start__ = (__ICFEDIT_region_ROM1_end__ + 1);
12 define symbol __ICFEDIT_region_ROM2_end__ = 0x0803F800 + 0x800 - 1;
13 define symbol __ICFEDIT_region_RAM_start__ = 0x20000000;
14 define symbol __ICFEDIT_region_RAM_end__ = 0x20007FFF;
15 /*-Sizes-*/
16 define symbol __ICFEDIT_size_cstack__ = 0x400;
17 define symbol __ICFEDIT_size_heap__ = 0x200;
18 /***** End of ICF editor section. ###ICF###*/
19
20 define memory mem with size = 4G;
21 define region ROM_region = mem:[from __ICFEDIT_region_ROM_start__ to __ICFEDIT_region_ROM_end__];
22 define region ROM1_region = mem:[from __ICFEDIT_region_ROM1_start__ to __ICFEDIT_region_ROM1_end__];
23 define region ROM2_region = mem:[from __ICFEDIT_region_ROM2_start__ to __ICFEDIT_region_ROM2_end__];
24 define region RAM_region = mem:[from __ICFEDIT_region_RAM_start__ to __ICFEDIT_region_RAM_end__];
25
26 define block CSTACK with alignment = 8, size = __ICFEDIT_size_cstack__ { };
27 define block HEAP with alignment = 8, size = __ICFEDIT_size_heap__ { };
28
29 initialize by copy { readwrite };
30 do not initialize { section .noinit };
31
32 place at address mem: __ICFEDIT_intvec_start__ { readonly section .intvec };
33
34
35 place in ROM_region { readonly };
36 place in ROM1_region { readonly section .MC_VectStoreAddr1 };
37 place in ROM2_region { readonly section .MC_VectStoreAddr2 };
38 place in RAM_region { readwrite,
39 block CSTACK, block HEAP };

```

3 Quick start guide

Follow the steps below to fine-tune a BLDC motor by using six-step square wave control method.

Step 1: Modify motor control drive mode and parameters according to motor parameters and application requirements (see Section 3.1).

Step 2: Connect with UI software (see Section 3.2).

Step 3: Select open loop control to check whether motor is running normally (see Section 3.3)

Step 4: Set Hall phase sequence self-learning (see Section 3.4)

Step 5: Current loop control parameter tuning.

(If current loop is not required, enable "WITHOUT_CURRENT_CTRL" in Step 1, omit Step 5 and Step 6, and go to Step 7 to adjust speed voltage loop)

STEP 1 - Use an oscilloscope to determine the current sampling point (see Section 3.5)

STEP 2 - Automatic identification of motor winding parameters (see Section 3.6)

STEP 3 - Current PI controller parameter auto identification (see Section 3.6)

STEP 4 - IQ tune: fine-tuning Kp and Ki values of current loop (see Section 3.7)

Step 6: Speed control parameters tuning (see Section 3.8)

Step 7: Speed voltage control parameters tuning (see Section 3.9)

Step 8: By default, control commands are provided via UI software. To switch to external potentiometer control, adjust the control command source (see Section 3.10).

3.1 Modify motor control mode and parameters

- The *motor_control_drive_param.h* header file allows users to configure the motor drive mode, motor parameters, board parameters and controller parameters.
- Users can configure the desired drive mode according to their hardware and motor configuration, as detailed below.

1. Select to use an internal crystal.

Table 2. Macro definition setting (internal crystal)

Definition	Description
INTERNAL_CLOCK_SOURCE	Use an internal crystal (disabled by default)

2. Select a motor development board as needed (for example, AT32-Motor-EV has V1 and V2 versions).

Table 3. Macro definition setting (EVB version)

Definition	Description
AT_MOTOR_EVB_V1	AT-MOTOR-EVB V1.x (include mc_hwio_v1.h file)
AT_MOTOR_EVB_V2	AT-MOTOR-EVB V2.0 (include mc_hwio_v1.h file)

3. Enable complementarity between low-side MOS transistor switch and upper-side PWM as needed. It is recommended to enable this feature in sensorless mode.

Table 4. Macro definition setting (upper-side PWM complementarity)

Definition	Description
COMPLEMENT	Enable complementarity between low-side MOS transistor switch and upper-side PWM (enabled by default; comment to disable)

4. Enable the low-side MOS inverted output based on the gate driver model used in the hardware. For example, U3315 on AT32-Motor-EV supports inverted output; therefore, this feature is enabled by default; whereas FD6288Q on AT32M412-LV-Motor-EV does not support inverted output; therefore, this feature is disabled by default.

Table 5. Macro definition setting (low-side MOS inverted output)

Definition	Description
GATE_DRIVER_LOW_SIDE_INVERT	Enable low-side MOS inverted output (enabled by default for AT32-Motor-EV and disabled by default for AT32M412-LV-Motor-EV)

5. Enable EMF voltage compensate according to the motor configuration.

Table 6. Macro definition setting (EMF voltage compensate)

Definition	Description
EMF_COMPENSATE	EMF voltage compensate (disabled by default; uncomment to enable)

6. In this demo project, it is Hall sensor by default. There is no need to modify this macro definition, and it is reserved for identification by code.

Table 7. Macro definition setting (sensored/sensorless method)

Definition	Description
HALL_SENSORS	Hall sensor
SENSORLESS	Sensorless control mode (general-purpose)

7. Enable current loop control as needed.

Table 8. Macro definition setting (without current loop)

Definition	Description
WITHOUT_CURRENT_CTRL	Voltage control without current loop (disabled by default; uncomment to enable)

8. Enable low-speed voltage control as needed.

Table 9. Macro definition setting (low-speed voltage control)

Definition	Description
LOW_SPEED_VOLT_CTRL	Low-speed voltage control (enabled by default; comment to disable)

9. Automatic identification of motor winding parameters; enable to automatically detect motor winding RS, LS and calculate current loop PI parameters (see Section 3.6 for details).

Table 10. Macro definition setting (auto identification of motor winding parameters)

Definition	Description
MOTOR_PARAM_IDENTIFY	Automatic identification of motor winding parameters (enabled by default; comment to disable)

10. Enable or disable the Artery motor control UI tool according to user needs.

Table 11. Macro definition setting (Artery motor control UI tool)

Definition	Description
USE_MOTOR_MONITOR	Enable Artery motor control UI tool (enabled by default; comment to disable)

11. Enable or disable the current filtering function based on user needs.

Table 12. Macro definition setting (Current Filtering Function)

Definition	Description
CURRENT_LP_FILTER	Enable the current filtering function (enabled by default; if defined, this function is activated).

- Table 13 gives the motor parameters used in the demo, such as number of pole-pairs, encoder parameters and Hall sensor parameters.

Table 13. Motor parameter definition

Definition	Description
RS_LL	Motor line-to-line resistance (unit: Ω)
LS_LL	Motor line-to-line inductance (unit: H)
POLE_PAIRS	Number of pole-pairs
KE	Motor KE value
NOMINAL_CURRENT	Nominal current (unit: ampere)
HALL_LEARN_DIR	Motor rotation direction after HALL self-learning
HALL_LEARN_0_STATE	The first state after HALL self-learning (BLDC: output V-shunt high-side PWM and W-shunt low-side ON)
HALL_LEARN_1_STATE	The second state after HALL self-learning (BLDC: output V-shunt high-side PWM and U-shunt low-side ON)
HALL_LEARN_2_STATE	The third state after HALL self-learning (BLDC: output W-shunt high-side PWM and U-shunt low-side ON)
HALL_LEARN_3_STATE	The fourth state after HALL self-learning (BLDC: output W-shunt high-side PWM and V-shunt low-side ON)
HALL_LEARN_4_STATE	The fifth state after HALL self-learning (BLDC: output U-shunt high-side PWM and V-shunt low-side ON)
HALL_LEARN_5_STATE	The sixth state after HALL self-learning (BLDC: output U-shunt high-side PWM and W-shunt low-side ON)

3.2 Connection settings

After the hardware and software are well prepared, set up connection between UI and the control board as follows (see AN0063 for details).

STEP-1

Connect the motor, AT-Link/J-Link and power supply to the motor control evaluation board, and connect USART interface to PC via an USB cable.

STEP-2

Use MDK to compile demo project code, and use J-Link or AT-Link to download to the on-board chip.

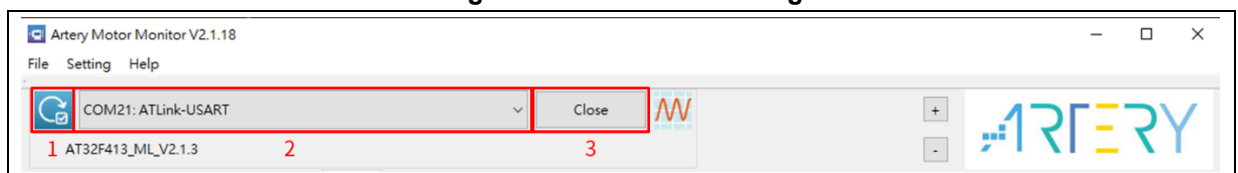
STEP-3

Run ArteryMotorMonitor_V2.1.x.exe (software version: V2.1.x); click File -> Open Project and select ArteryMotorMonitor_V2.1.x.atmcx-> Open.

STEP-4

Click the update icon (1.) of Serial Port and select the corresponding serial port (2.); then click Open(3.) to enable real-time communication, as shown in Figure 5.

Figure 5. Connection setting

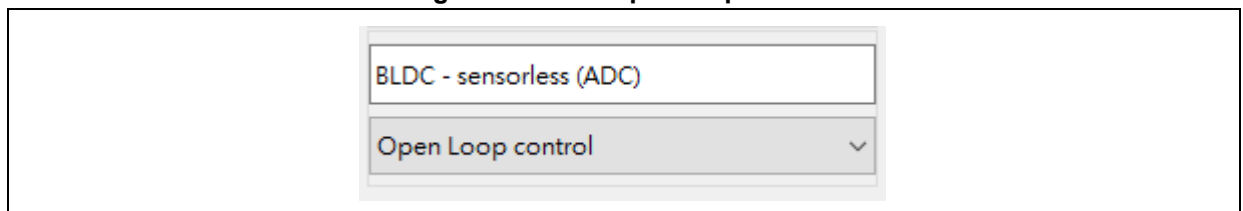


3.3 Six-step square wave open loop control

Using open-loop voltage control, this mode allows motor rotation without position sensors. It serves to confirm the motor's operational status and direction or to adjust sensorless open loop startup parameters for BLDC. The open loop voltage and speed are incremented according to the motor running speed and motor phase current value.

STEP-1: Select "Open Loop control".

Figure 6. Select open loop control

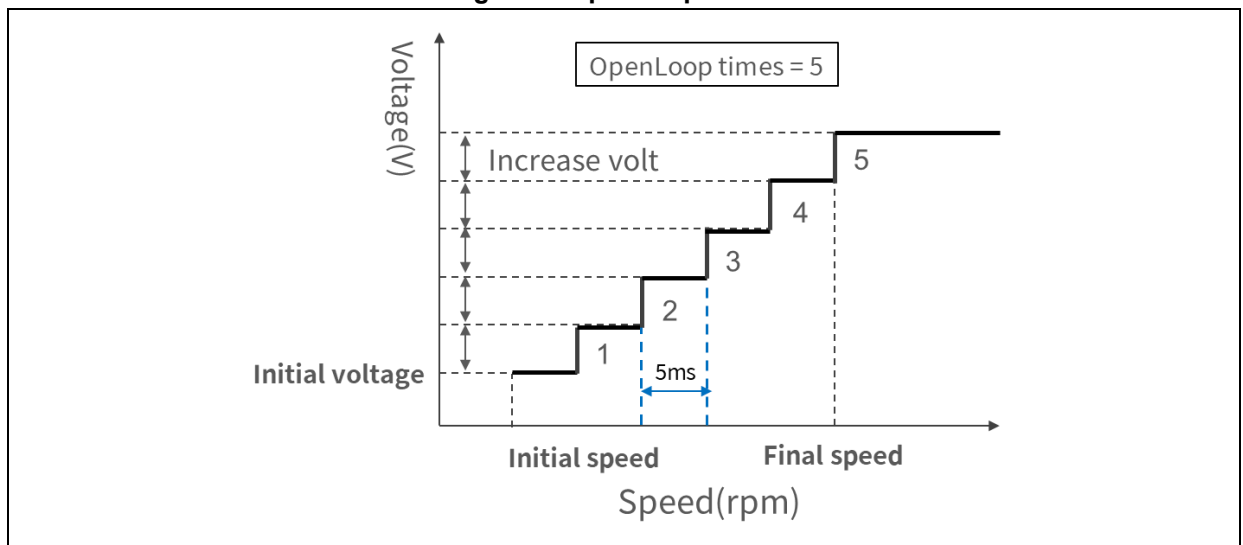


STEP-2: Go to "Tuning Parameters" (as shown in Figure 7) and set parameters in "OpenLoop initial voltage", "OpenLoop initial speed", "OpenLoop final speed", "OpenLoop times" and "OpenLoop increase volt". Figure 8 shows the open loop control.

Figure 7. Open loop control parameters (BLDC)

Basic Tuning Parameters		
Open Loop Control		
OpenLoop initial voltage	0.75	V
OpenLoop initial speed	100	rpm
OpenLoop increase volt	0.004	V
OpenLoop final speed	350	rpm
OpenLoop times	90	

Figure 8. Open loop control



Tips:

- Initially, set **Openloop increase volt** and **Openloop times** to 0, and set **Openloop final speed** and **Openloop initial speed** to the same value. Then find the appropriate starting voltage and speed.
- Set the final speed according to the initial speed, and adjust **increase volt** and **Openloop times**. In case of underpower, increase **increase volt** value or **Openloop times** value (it is recommended to be >50).

STEP-3: Click on “Start Motor”, and monitor the current until the motor starts to run properly (set the open loop voltage value appropriately to prevent overheating damage to the motor).

STEP-4. Fill in the parameters into macro definitions in the *motor_control_drive_param.h* file, as shown in Figure 9, and recompile and re-program the code.

Figure 9. Macro definition of open loop control parameters

```

motor_control_drive_param.h
225  /* open loop control */
226  #define OLC_INIT_SPD           (100)      /*!< rpm */
227  #define OLC_FINAL_SPD         (350)      /*!< rpm */
228  #define OLC_TIMES              (90)       /*!< Accumula
229  #define OLC_INIT_VOLT          (0.75)     /*!< V */
230  #define OLC_VOLT_INC           (0.004)

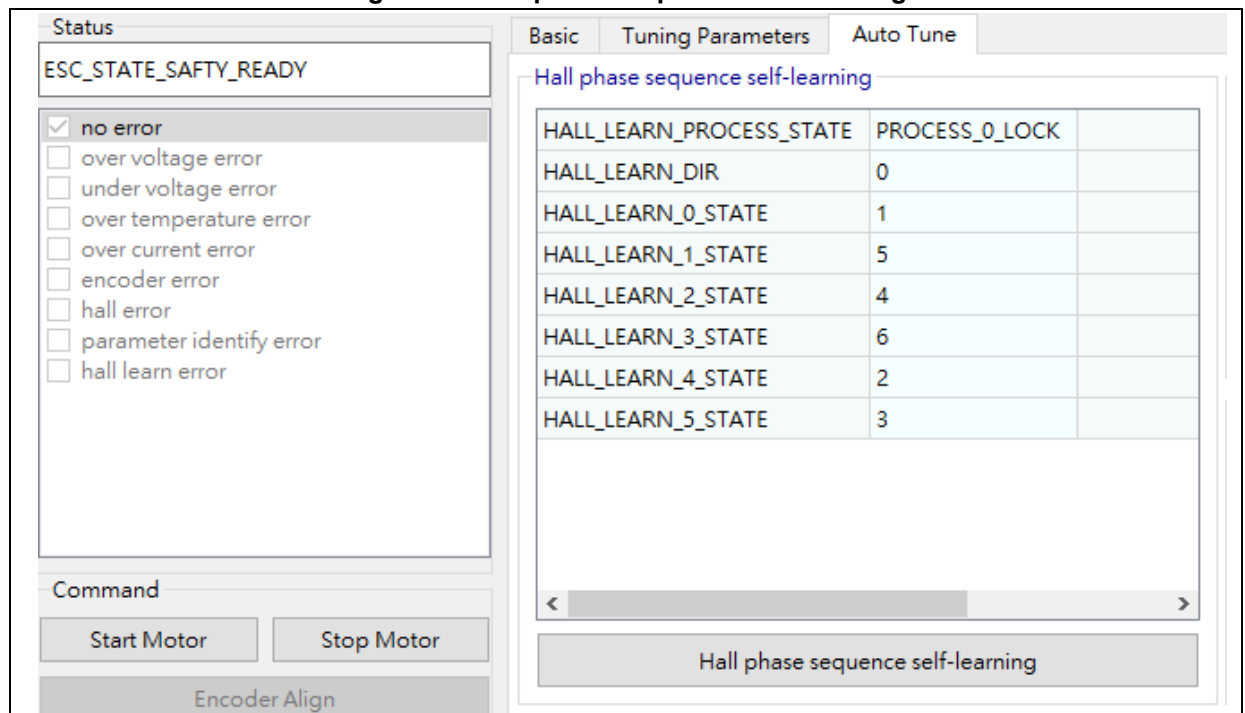
```

3.4 Hall phase sequence self-learning

STEP-1: Connect to UI interface.

STEP-2: Enter debug code and go to “Auto Tune” window; click on “Hall phase sequence self-learning” key in ESC_STATE_SAFETY_READY status to execute Hall phase sequence self-learning, as shown in Figure 10.

Figure 10. Hall phase sequence self-learning



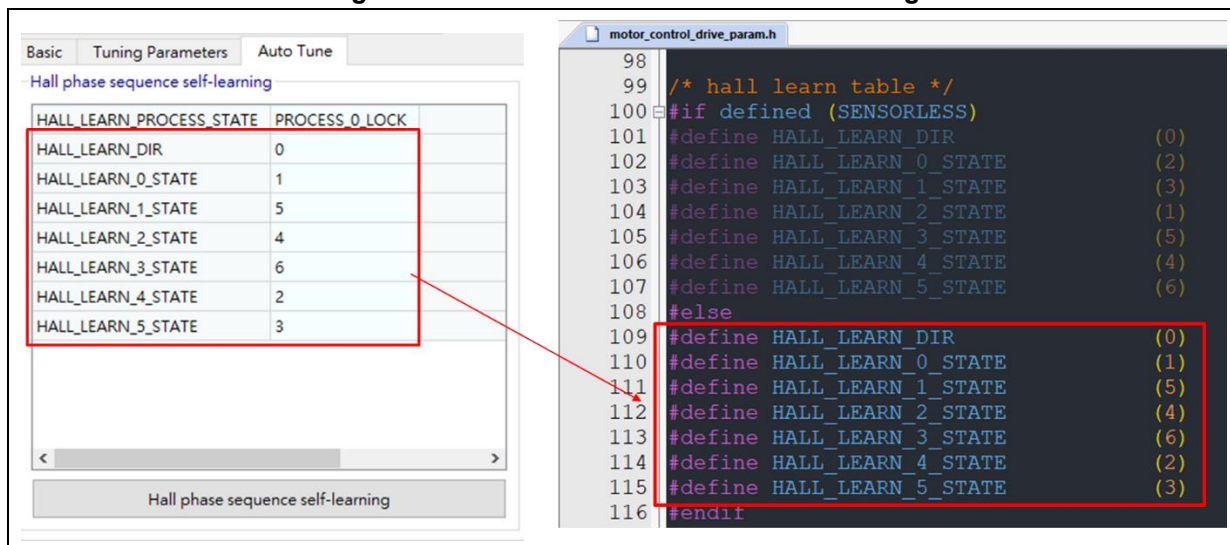
STEP-3: The motor runs in open loop mode. If the motor is running in the wrong direction, wait for the motor to stop and then click on “Hall phase sequence self-learning” button again to reverse the direction. Self-learning is complete when the motor stops.

Note: If the motor does not run smoothly or does not run at all, go back to the open loop control to adjust the open loop voltage and speed, then re-compile and re-program, and restart STEP-2.

STEP-4: Check whether the `HALL_LEARN_PROCESS_STATE` is “`PROCESS_4_FINISH`”.

STEP-5: Fill the values of `HALL_LEARN_DIR` and `HALL_LEARN_0_STATE~HALL_LEARN_5_STATE` to the “hall learn table” macro definition (`motor_control_drive_param.h`) (Figure 11), and then re-compile and re-program code.

Figure 11. Macro definition of Hall self-learning



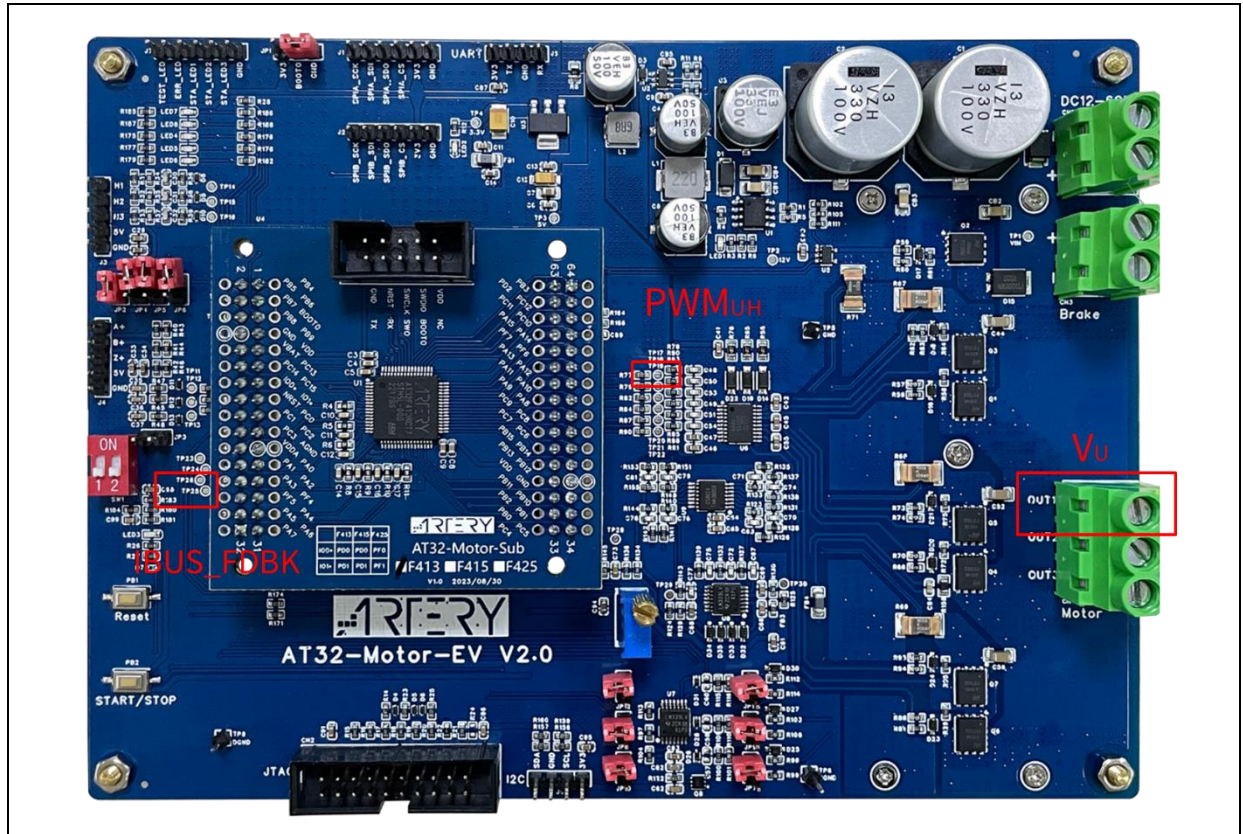
3.5 Current sampling point

Use a 3-channel (at least) oscilloscope to determine the current sampling point. This is crucial for accurate current measurement and control. Signals to be measured are shown in Figure 12.

Oscilloscope channel	Measuring position	Example: AT32-Motor-EVB2.0
CH1	U-shunt output voltage	CN4 OUT1
CH2	Current sensing signal *note [3]	IBUS_FDBK(TP26)
CH3	PWM UH output signal (MCU side)	TIM1_CH1(TP17)

* Note [3]: Because of the need to sense positive and negative currents, the current sense signal has a DC level when the current is 0A. For example, AT32-Motor-EVB2.0 is 0.833V, and CH2 needs to be set to eliminate the zero (-0.83V) when the signal is measured.

Figure 12. Current sampling point signal measurement position (AT32-Motor-EVB2.0)



STEP-1: Select six-step square wave open loop control mode to run motor.

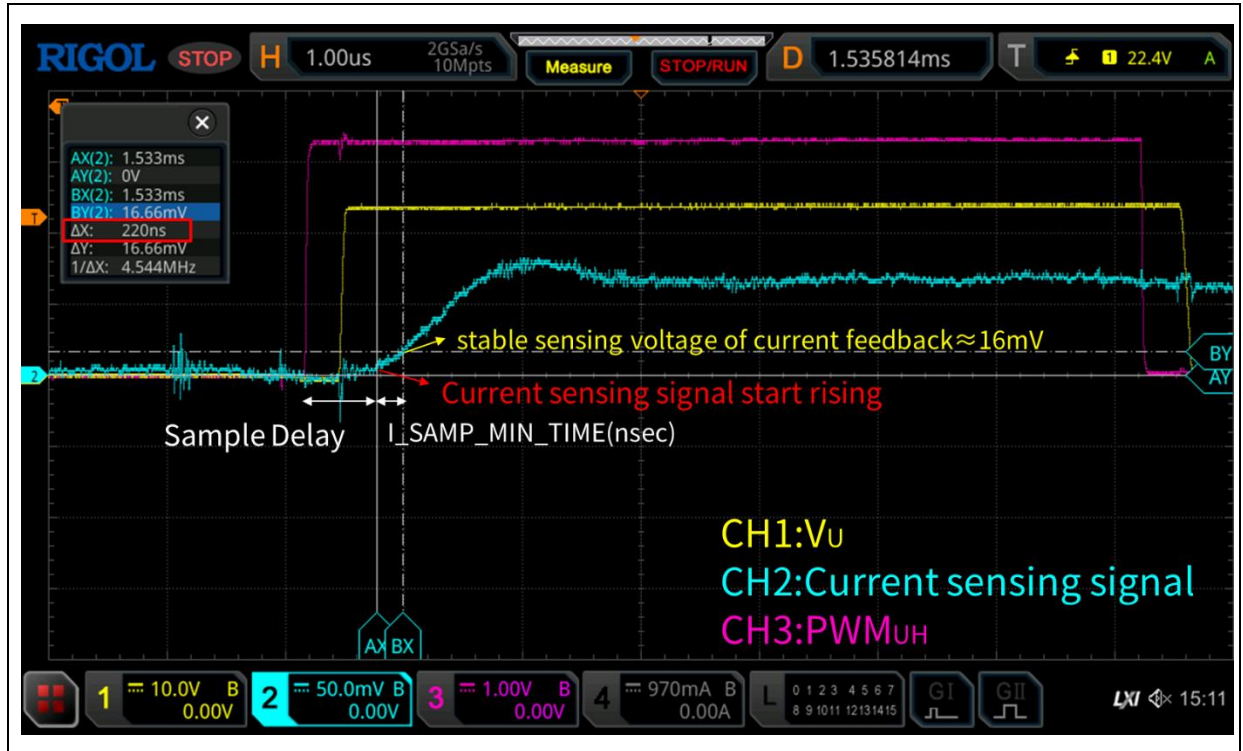
STEP-2: The rising edge of PWM_UH is used as the trigger point to measure the current sampling delay time, as shown in Figure 13.

Sample Delay = Time difference between PWM UH rise time and the time current sensing signal starts to rise (unit: usec)

$$I_SAMP_DELAY(nsec) = Sample\ Delay(nsec) + Dead\ time(nsec)$$

STEP-3: Move the cursor to a stable ADC sampling voltage (about 16 mV), and measure the time required for CH2 current sensing signal from 0 mV to 16 mV, that is, I_SAMP_MIN_TIME. As shown in Figure 13, the time is 220 nsec.

Figure 13. Current sampling delay measurement waveform



STEP-4: Fill the delay time value measured in STEP-3 into the macro definition

"I_SAMPLE_MIN_TIME" in *motor_control_drive_param.h* file, as shown in Figure 14, and then re-compile and re-program code.

Figure 14. Macro definition of current sampling point

```

196 /* I-SAMPLE PARAMETER */
197 #define I_SAMP_MIN_TIME (220) /*!< nsec */
198 #define I_SAMP_DELAY (500) /*!< nsec */

```

3.6 Auto identification and tuning of parameters

Automatic parameter identification is divided into two parts, namely automatic identification of motor coil parameters and self-tuning of current PI controller parameters.

Auto identification of motor winding parameters are not required if there are RS_LL and LS_LL parameters. Users can directly modify macro definitions of RS_LL and LS_LL in *motor_control_drive_param.h*.

Auto identification of motor winding parameters:

STEP-1: Fill the macro definition NOMINAL_CURRENT into *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 15.

Figure 15. Macro definition of motor nominal current

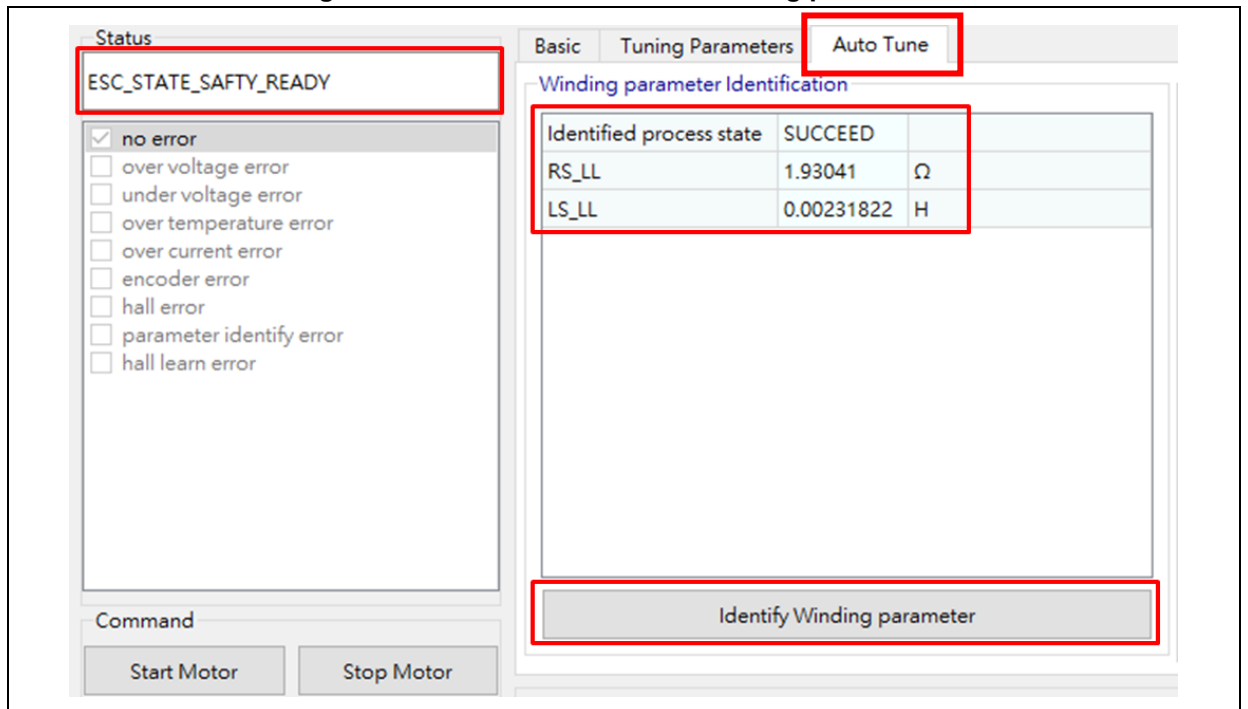
```

motor_control_drive_param.h
79 /***** Motor-related parameter *****/
80 #define POLE_PAIRS (8/2)
81 #define RS_LL (1.89f) /* Stator resistance (line-to-line) */
82 #define LS_LL (0.002387f) /* Stator inductance (line-to-line) */
83
84 #define NOMINAL_CURRENT (1.7f)
85

```

STEP-2: Go to UI --> Auto Tune, and click the “Identify Winding parameter” key in “safety ready” status to perform winding parameter identification, as shown in Figure 16.

Figure 16. Auto identification of winding parameters



STEP-3: Check and confirm “SUCCEED” of “Identified process state”; get the RS_LL and LS_LL and fill them into the corresponding macro definitions in *motor_control_drive_param.h* file, as shown in Figure 17.

Figure 17. Macro definitions of winding parameters

```

motor_control_drive_param.h
89
90 /***** Motor-related parameter *****/
91 /* Motor parameters */
92 #define RS_LL (1.89) /* Stator resistance, ohm */
93 #define LS_LL (0.002387) /* Stator inductance, H */
94

```

Current PI controller parameter auto tuning:

STEP-1: Fill in the macro definition of input DC voltage (VDC_RATED) into *motor_control_drive_param.h* file, and re-compile and re-program, as shown in Figure 18.

Figure 18. Motor rated voltage

```

motor_control_drive_param.h
100
101 /***** Drive-related parameter *****/
102 /* basic */
103 #define VDC_RATED (24.0f)
104 #define V_SENSE_GAIN (10/(3.9f+180+10)) // 0.05157
105 #define ADC_REFERENCE_VOLT (3.3f)
106 #define ADC_DIGITAL_SCALE_12BITS (4095)
    
```

STEP-2: Go to Auto Tune window, and click the “Auto-tune Current PI parameter” button in “safety ready” status to perform auto tuning of current PI controller parameters. After the auto tuning is completed, current Q axis PI controller parameters are automatically updated, as shown in Figure 19.

Figure 19. Current PI controller parameter auto tuning

Auto-tune PI parameter of Current controller

Torque KP	7717
Torque KI	6110
Torque KP DIV	512
Torque KI DIV	8192

Auto-tune Current PI parameter

STEP-3: Fill the parameters shown in Figure 19 into the corresponding macro definition (motor_control_drive_param.h), then recompile and re-program code, as shown in Figure 20.

Figure 20. Macro definition of current PI controller parameters

```

motor_control_drive_param.h
242 /* pi parameter */
243 #define PID_IS_KP_DEFAULT 2000
244 #define PID_IS_KI_DEFAULT 200
245 #define PID_IS_KP_DIV 256
246 #define PID_IS_KP_DIV_LOG LOG2(PID_IS_KP_DIV)
247 #define PID_IS_KI_DIV 1024
248 #define PID_IS_KI_DIV_LOG LOG2(PID_IS_KI_DIV)
    
```

Conversion formula for older versions of motor control library (MC_Library_Project_V2.1.1 and older version):

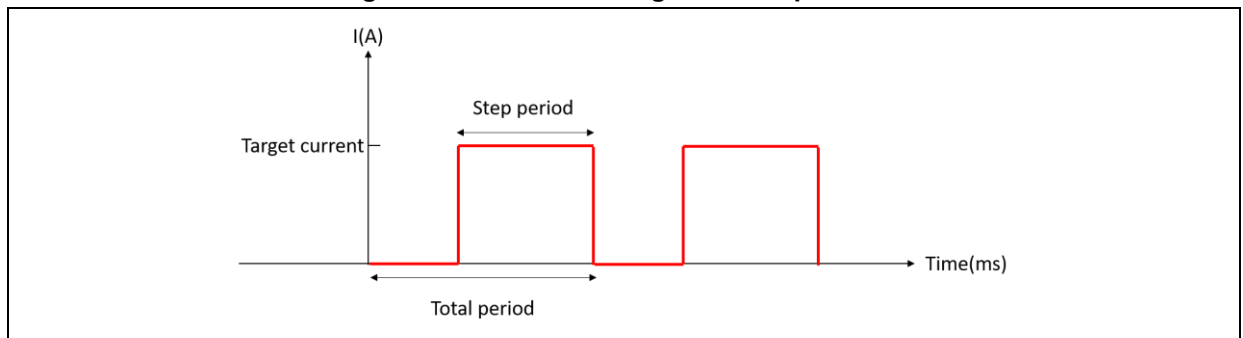
$$PID_IS_KP_DIV_LOG = \log_2 \text{Torque } KP \text{ DIV}$$

$$PID_IS_KI_DIV_LOG = \log_2 \text{Torque } KI \text{ DIV}$$

3.7 IQ tune

A step current is generated in IQ Tune mode, as shown in Figure 21. Parameters related to the step current are adjustable. The step current is generated to help check the current response after adjusting PID parameters of Q axis current.

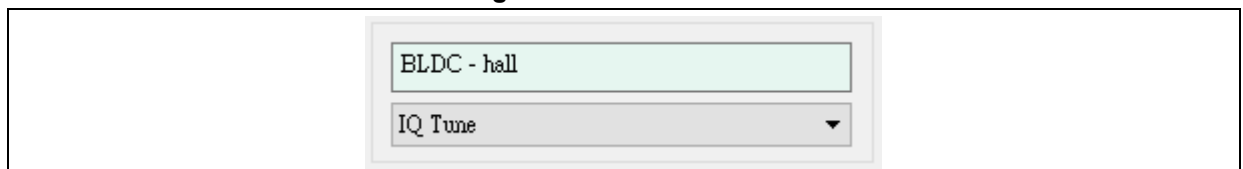
Figure 21. Schematic diagram of step current



Follow the below steps:

STEP-1: Select "IQ tune".

Figure 22. Select IQ tune



STEP-2: Go to Tuning Parameters window, and set PID parameters and step current parameters. Parameters obtained by current PI controller parameters auto tuning can be used for initial adjustment, and users can adjusted these parameters as needed to get the desired current response, as shown in Figure 23.

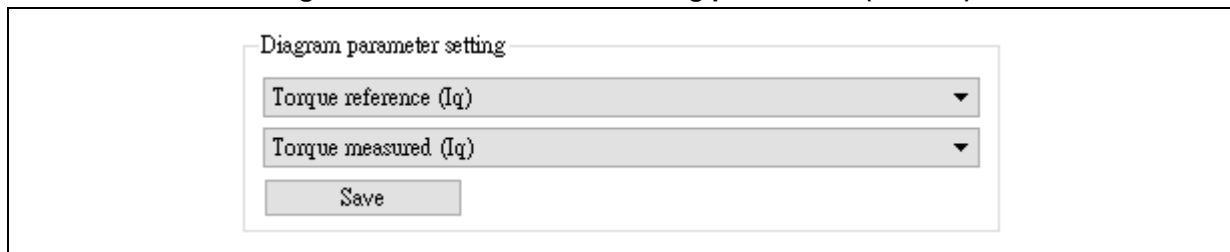
Figure 23. PID parameters and step current parameters

Basic			Tuning Parameters		
Unit step config			IQ Current control		
Current Tune target current	0.999	A	Torque KP	25000	
Current Tune total period	100	ms	Torque KI	3000	
Current Tune step period	2	ms	Torque KP DIV	2048	
			Torque KI DIV	4096	

STEP-3: Click on “Start Motor”.

STEP-4: Set “Torque reference(Iq)” and “Torque measured(Iq)” in “Diagram parameter setting”, and then click on “Save”.

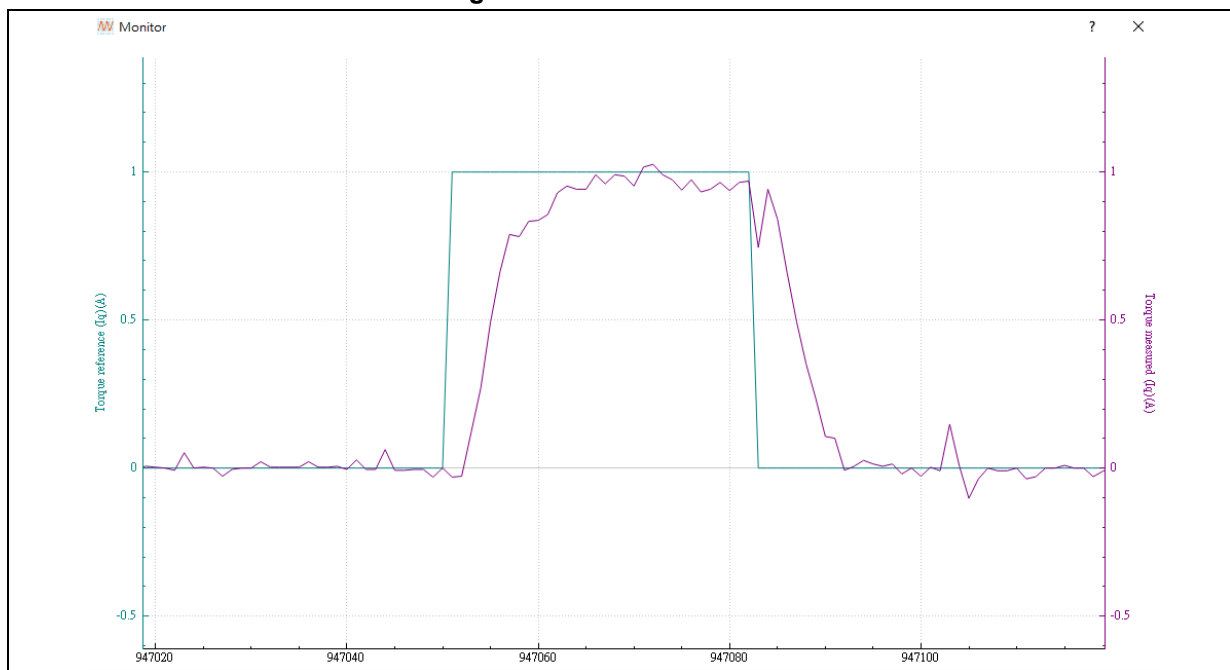
Figure 24. Set channel monitoring parameters (IQ tune)



STEP-5: Click the drawing icon  to open the waveform window. Refer to *AN0063 AT32 Motor Monitor Application Note* for details.

STEP-6: Check whether the current response is as expected, as shown in Figure 25. If not, click to stop the motor and repeat STEP-2~STEP-6.

Figure 25. IQ tune waveform



STEP-7: After the IQ tuning is completed, fill parameters into the macro definition (motor_control_drive_param.h); then recompile and re-program code, as shown in Figure 26.

Figure 26. Macro definition of current PI controller parameters

```

242  /* pi parameter */
243  #define PID_IS_KP_DEFAULT      2000
244  #define PID_IS_KI_DEFAULT      200
245  #define PID_IS_KP_DIV          256
246  #define PID_IS_KP_DIV_LOG      LOG2 (PID_IS_KP_DIV)
247  #define PID_IS_KI_DIV          1024
248  #define PID_IS_KI_DIV_LOG      LOG2 (PID_IS_KI_DIV)

```

3.8 Speed loop control

In this mode, users can set the speed PID parameters, acceleration and deceleration, and check the response through waveform.

Follow the below steps:

STEP-1: Select “Speed Control”.

STEP-2: Go to Tuning Parameters window, and set speed PID parameters, acceleration and deceleration.

Figure 27. PID parameters, acceleration and deceleration

Speed control(With current control loop)

Speed KP	1000	
Speed KI	20	
Speed KP DIV	4096	
Speed KI DIV	32768	
Speed acceleration	5	rpm/ms
Speed deceleration	5	rpm/ms

STEP-3: Select “software control” as the control source and set the target speed (Speed reference).

Figure 28. Set target speed

Target speed

Speed reference 0 rpm

STEP-4: Click on “Start Motor”.

STEP-5: Set “Speed reference” and “Speed measured” in “Diagram parameter setting”.

Figure 29. Set channel monitoring parameters (speed control)

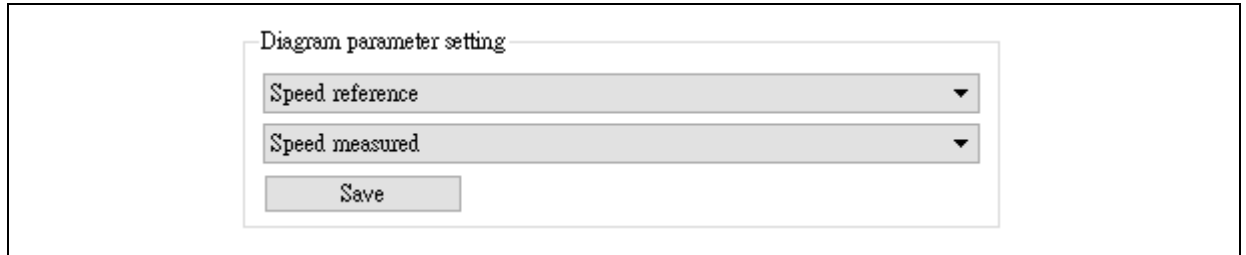


Diagram parameter setting

Speed reference ▼

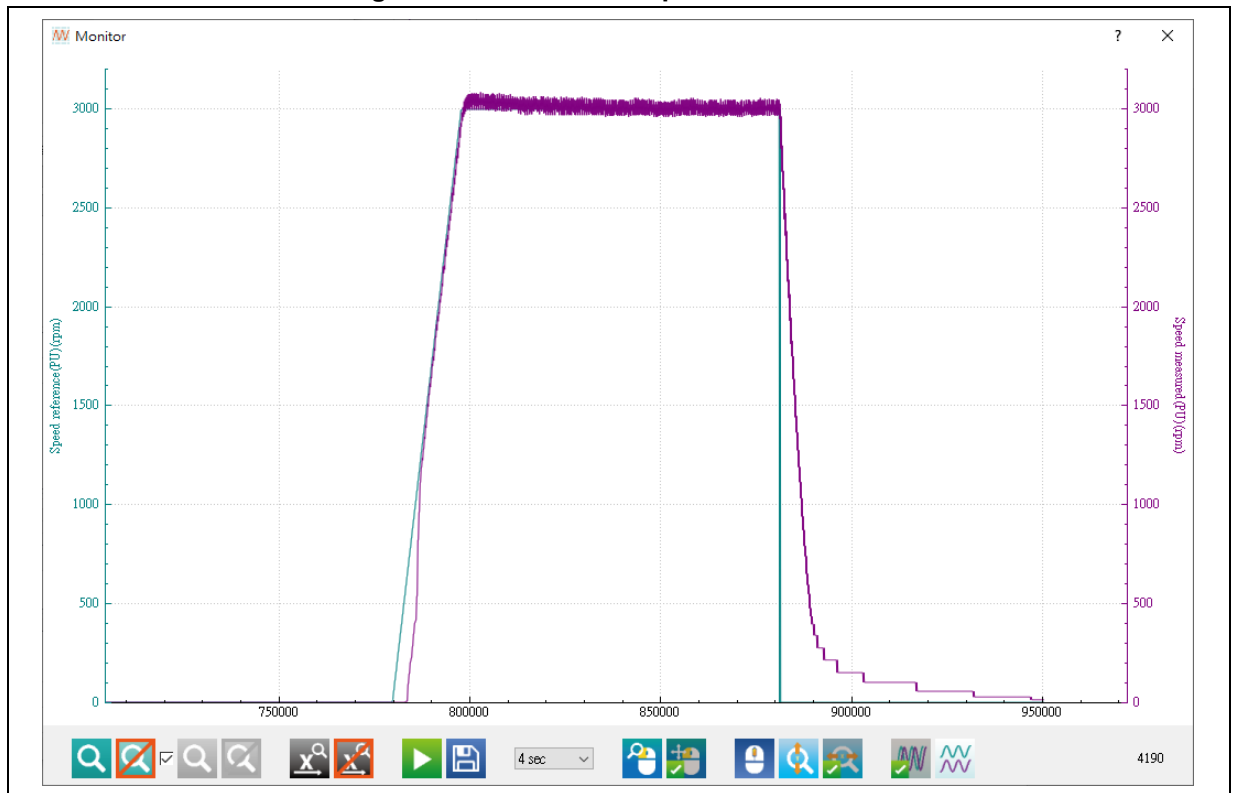
Speed measured ▼

Save

STEP-6: Click on the drawing icon  to open the waveform window.

STEP-7: Check whether the speed response is as expected, as shown in Figure 30. If not, click to stop the motor and repeat STEP-2~STEP-7.

Figure 30. Waveform in speed control mode



3.9 Speed voltage control

The current is low when the motor runs at a low speed. This demo project provides a low-speed voltage control method to support voltage control at low speed running and switches to closed-loop control when the current becomes high.

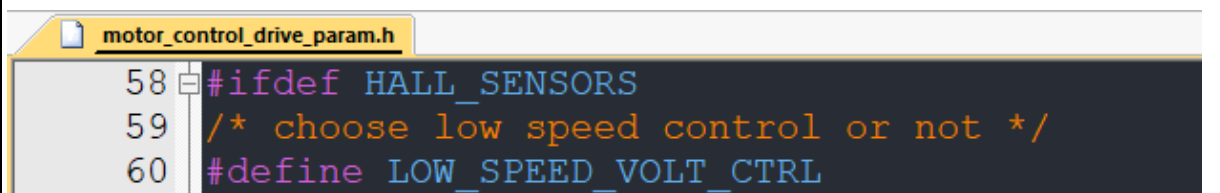
The speed voltage control method is used to control output voltage according to the speed difference. It is required when the WITHOUT_CURRENT_CTRL (voltage control only) is enabled or the low-speed range of LOW_SPEED_VOLT_CTRL (low-speed voltage control) is enabled.

In this mode, users can set the speed voltage PID parameters, and check the response through waveform.

Follow the below steps.

STEP-1: Define LOW_SPEED_VOLT_CTRL in *motor_control_drive_param.h* file, as shown in Figure 31.

Figure 31. Definition of low-speed voltage control



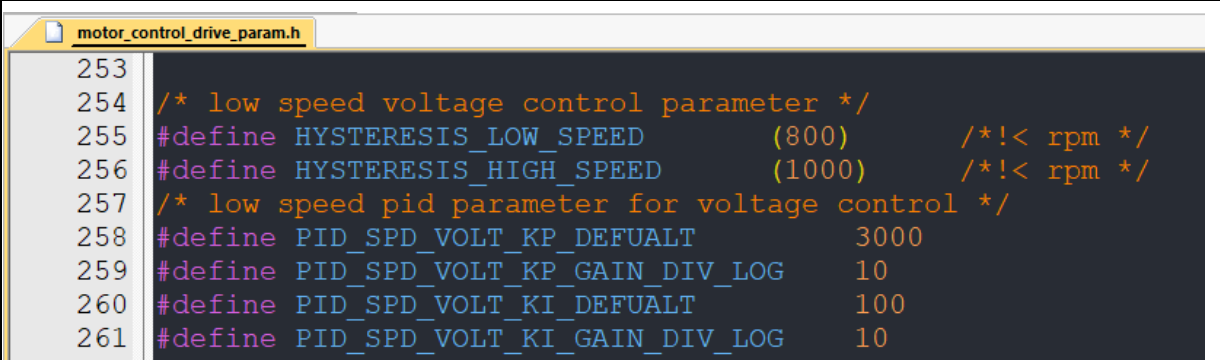
```

58 #ifdef HALL_SENSORS
59 /* choose low speed control or not */
60 #define LOW_SPEED_VOLT_CTRL

```

STEP-2: Set the minimum speed (HYSTERESIS_LOW_SPEED) shifting from current control to low-speed voltage control, and the maximum speed (HYSTERESIS_HIGH_SPEED) shifting from low-speed voltage control to current control, as shown in Figure 32.

Figure 32. Low-speed voltage control parameter definition



```

253
254 /* low speed voltage control parameter */
255 #define HYSTERESIS_LOW_SPEED      (800)      /*!< rpm */
256 #define HYSTERESIS_HIGH_SPEED    (1000)     /*!< rpm */
257 /* low speed pid parameter for voltage control */
258 #define PID_SPD_VOLT_KP_DEFAULT   3000
259 #define PID_SPD_VOLT_KP_GAIN_DIV_LOG 10
260 #define PID_SPD_VOLT_KI_DEFAULT   100
261 #define PID_SPD_VOLT_KI_GAIN_DIV_LOG 10

```

STEP-3: Select “Speed Control”.

STEP-4: Go to Tuning Parameters window and set the speed voltage PID parameters.

Figure 33. Speed voltage controller PID parameter setting

Speed Voltage Controller(Without Current control loop)

Speed Voltage KP	3000	
Speed Voltage KI	100	
Speed Voltage KP DIV	1024	
Speed Voltage KI DIV	1024	

STEP-5: Select “software control” as the control source and set the target speed (Speed reference), for example, 300 rpm.

Figure 34. Set target speed

Target speed

Speed reference 0 rpm

STEP-6: Click on “Start Motor”.

STEP-7: Set “Speed reference(PU)” and “Speed measured(PU)” in “Diagram parameter setting”, and then click on “Save”.

Figure 35. Set channel monitoring parameters (speed control)

Diagram parameter setting

Speed reference(PU) ▾

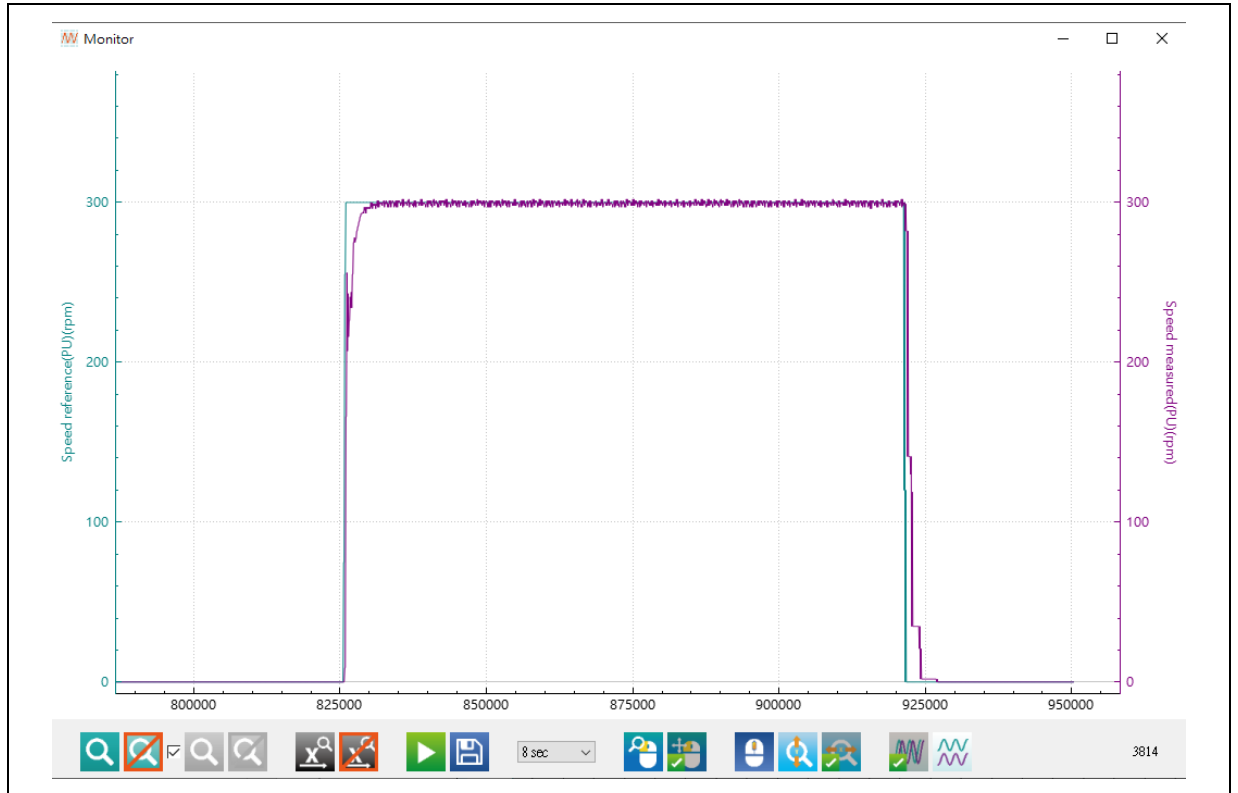
Speed measured(PU) ▾

Save

STEP-8: Click the drawing icon to open the waveform window.

STEP-9: Check whether the speed response is as expected, as shown in Figure 36. If not, click to stop the motor and repeat STEP-4~STEP-9.

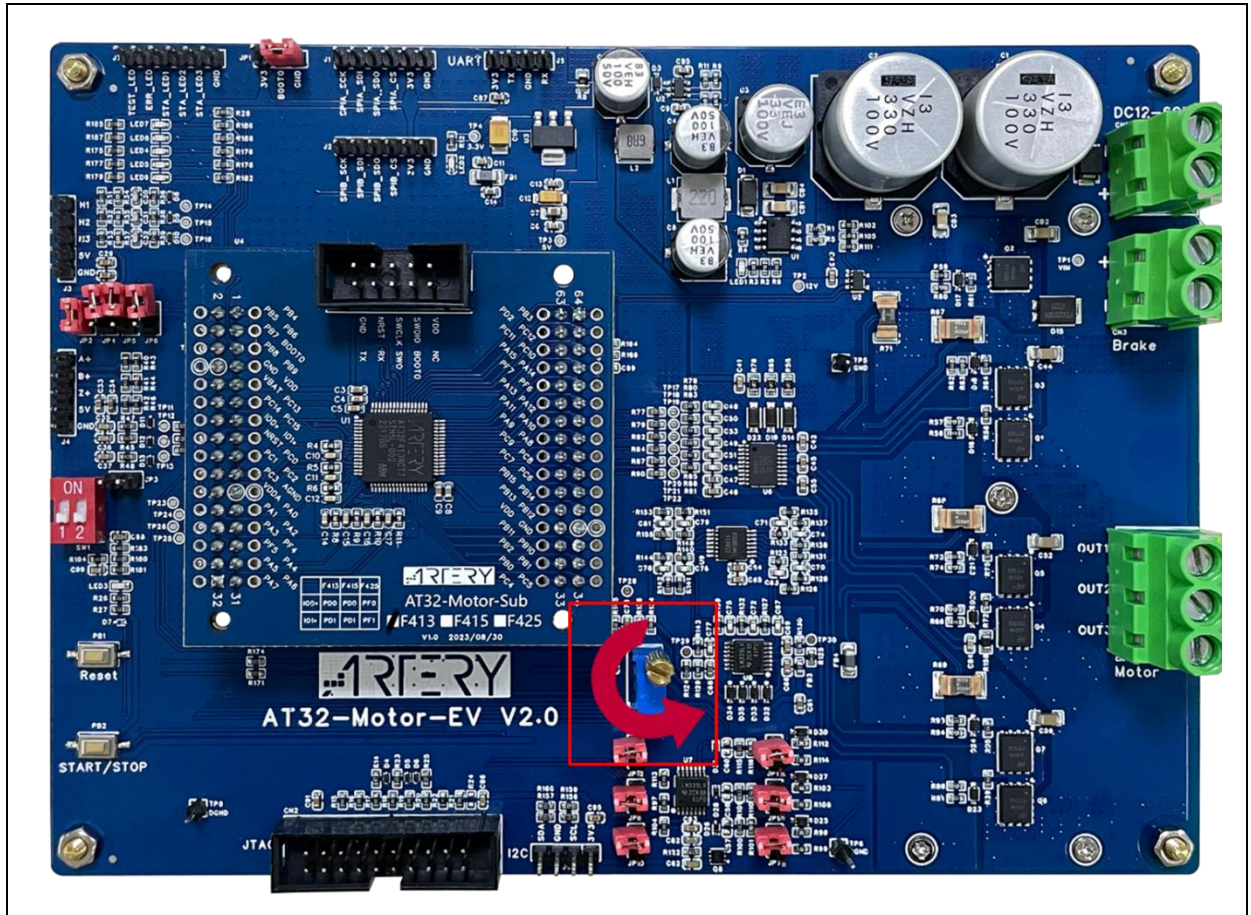
Figure 36. Waveform in speed voltage control mode



3.10 External voltage control/software command control

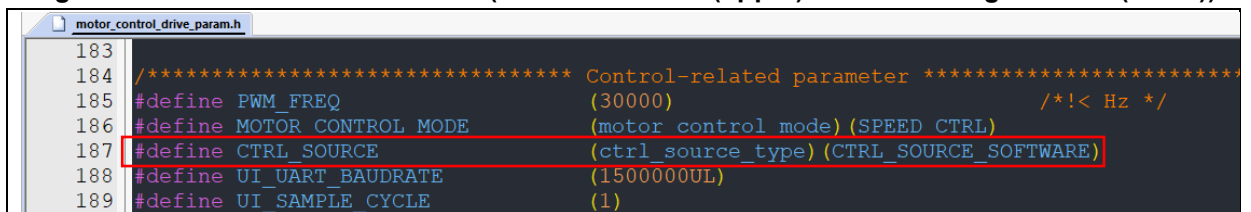
This demo project support two control sources, i.e., external voltage control and software control. The external voltage control mode adjusts speed or torque through external voltage. Users can adjust the potentiometer (as shown in the red box below) to control the command value. The voltage increases when the potentiometer is turned **counterclockwise**. The software control mode inputs speed or torque command value for control purpose. In this demo project, software control mode is set by default.

Figure 37. External voltage control – potentiometer



Users can select the control source by modifying the corresponding definition (CTRL_SOURCE in *motor_control_drive_param.h* file, as shown in Figure 38) or through PC software (see Figure 39). Control parameters are different for different control modes, such as the target speed (as shown in Figure 39) in speed control mode, or torque reference (as shown in Figure 40) in torque control mode.

Figure 38. Control source definition (software control (upper)/external voltage control (lower))



```

183
184 /***** Control-related parameter *****/
185 #define PWM_FREQ (30000) /*!< Hz */
186 #define MOTOR_CONTROL_MODE (motor_control_mode) (SPEED_CTRL)
187 #define CTRL_SOURCE (ctrl_source_type) (CTRL_SOURCE_EXTERNAL)
188 #define UI_UART_BAUDRATE (1500000UL)
189 #define UI_SAMPLE_CYCLE (1)

```

1) External voltage control

Go to “Control source” and select “external voltage control” to adjust the speed or torque through external voltage. The target speed or torque reference field displays the converted speed or torque at the current control voltage.

Note: This field cannot be modified in the external control source mode.

Figure 39. Speed control mode (external voltage control)

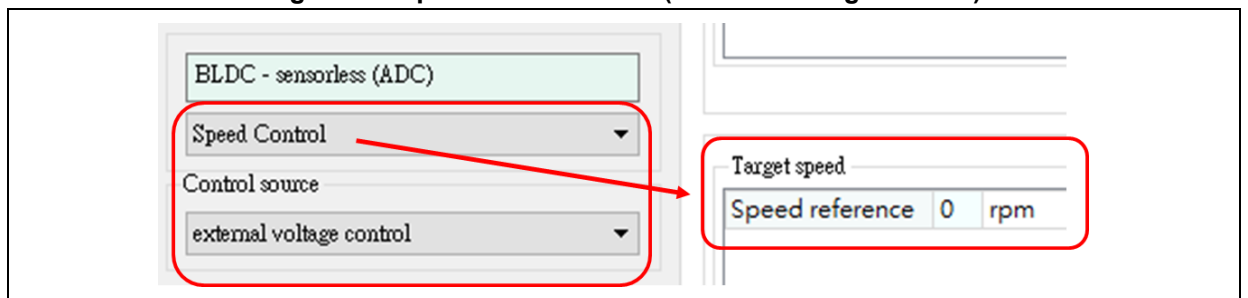
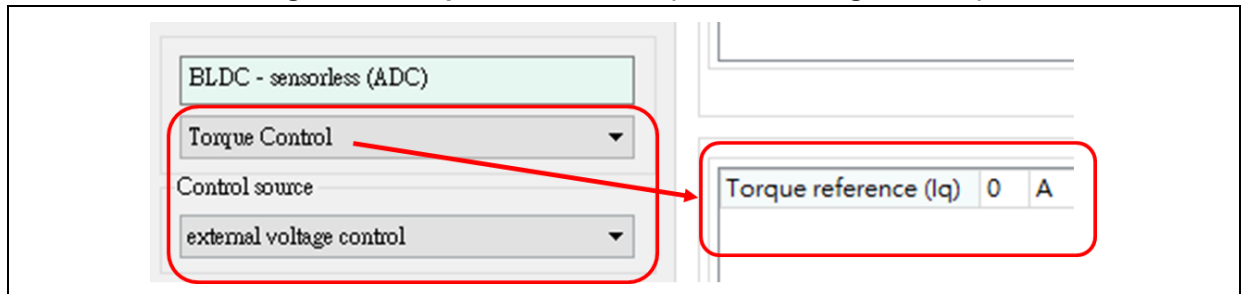


Figure 40. Torque control mode (external voltage control)



2) Software control

By default, the software control mode is enabled. You can switch to software control mode by selecting "Software control" from the source selection dropdown menu, as shown in Figure 41. In this mode, the speed/torque of the motor is adjusted by modifying the target speed/torque via the UI interface. Double-clicking on this field allows you to change the value. Once the setting is complete, a message indicating the successful setting will be displayed in the field below.

Figure 41. Set target speed in software control mode

The screenshot displays the software control interface for the AT32 BLDC Hall. The 'Control source' is set to 'software control'. The 'Target speed' is set to 'Speed reference 500 rpm'. Below the settings, there is a table with columns 'Time' and 'Motor' (labeled 'Message' in the image). The table contains three entries:

	Time	Motor	Message
1	16:49:32		Set REG 10 = 500:OK
2	16:49:26		Set REG 10 = 500:OK
3	16:49:22		Set REG 9 = 0:OK

4 Revision history

Table 14. Document revision history

Date	Version	Revision note
2024.05.17	2.1.0	Initial release.
2024.12.27	2.1.1	Updated Flash and ROM configurations, updated speed voltage control and updated some definitions.
2025.10.09	2.1.2	Modified Flash ROM configuration in Table 1.

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services; ARTERY assumes no liability for purchasers' selection or use of the products and the relevant services.

No license, express or implied, to any intellectual property right is granted by ARTERY herein regardless of the existence of any previous representation in any forms. If any part of this document involves third party's products or services, it does NOT imply that ARTERY authorizes the use of the third party's products or services, or permits any of the intellectual property, or guarantees any uses of the third party's products or services or intellectual property in any way.

Except as provided in ARTERY's terms and conditions of sale for such products, ARTERY disclaims any express or implied warranty, relating to use and/or sale of the products, including but not restricted to liability or warranties relating to merchantability, fitness for a particular purpose (based on the corresponding legal situation in any unjudicial districts), or infringement of any patent, copyright, or other intellectual property right.

ARTERY's products are not designed for the following purposes, and thus not intended for the following uses: (A) Applications that have specific requirements on safety, for example: life-support applications, active implant devices, or systems that have specific requirements on product function safety; (B) Aviation applications; (C) Aerospace applications or environment; (D) Weapons, and/or (E) Other applications that may cause injuries, deaths or property damages. Since ARTERY products are not intended for the above-mentioned purposes, if purchasers apply ARTERY products to these purposes, purchasers are solely responsible for any consequences or risks caused, even if any written notice is sent to ARTERY by purchasers; in addition, purchasers are solely responsible for the compliance with all statutory and regulatory requirements regarding these uses.

Any inconsistency of the sold ARTERY products with the statement and/or technical features specification described in this document will immediately cause the invalidity of any warranty granted by ARTERY products or services stated in this document by ARTERY, and ARTERY disclaims any responsibility in any form.

© 2025 ARTERY Technology – All Rights Reserved